

## 개요

이 문서는 CSCI89 Lecture 9 강의 내용을 바탕으로, 자연어 처리(NLP)에 적용되는 다양한 고전적 머신러닝 기법들을 다룹니다. 주요 목표는 나이브 베이즈, K-최근접 이웃, 로지스틱 회귀, 서포트 벡터 머신, 결정 트리, 랜덤 포레스트와 같은 모델들의 기본 원리, 장단점, 그리고 NLP 문제에 대한 적용 방법을 이해하는 것입니다. 특히, 텍스트 데이터를 벡터 형태로 변환하는 과정과, 모델의 성능을 평가하고 최적화하는 교차 검증 및 하이퍼파라미터 튜닝 기법에 중점을 둡니다. 강의에서 다룬 STM(Structural Topic Modeling) 복습부터 시작하여, 실제 BBC 뉴스 데이터셋을 활용한 텍스트 분류 실습을 통해 각 모델의 작동 방식과 성능을 비교 분석합니다. 이 문서는 시험 대비 및 실습을 위한 독립적인 학습 자료로 활용될 수 있도록 상세한 설명과 코드 예시를 포함합니다.

## 용어 정리

다음 표는 강의에서 다루는 주요 용어들을 정리한 것입니다.

| 용어                   | 쉬운 설명                                                  | 원어                                        | 비고                              |
|----------------------|--------------------------------------------------------|-------------------------------------------|---------------------------------|
| STM                  | 토픽 모델링에 문서 메타데이터를 통합하여 토픽의 분포와 내용을 분석하는 방법             | Structural Topic Modeling                 | LDA의 확장 버전                      |
| TF-IDF               | 단어의 중요도를 측정하는 방법. 문서 내 빈도와 전체 문서에서의 희귀도를 반영            | Term Frequency-Inverse Document Frequency | 텍스트를 수치 벡터로 변환                  |
| Naive Bayes          | 베이즈 정리를 기반으로 하며, 특징들이 서로 독립이라는 '나이브한' 가정을 사용하는 분류기     | Naive Bayes Classifier                    | 스팸 탐지, 감성 분석에 유용                |
| KNN                  | 새로운 데이터 포인트를 가장 가까운 K 개의 이웃들의 정보(클래스 또는 값)를 기반으로 분류/예측 | K-Nearest Neighbors                       | 거리 측정 및 K값 선택 중요                |
| Logistic Regression  | 선형 모델의 출력을 시그모이드 함수를 통해 0과 1 사이의 확률로 변환하여 분류하는 모델      | Logistic Regression                       | 이진 분류에 주로 사용                    |
| SVM                  | 두 클래스 사이의 마진(경계)을 최대화하는 결정 경계를 찾는 분류기                  | Support Vector Machines                   | 커널 트릭으로 비선형 분류 가능               |
| Decision Tree        | 데이터를 일련의 질문(분할 기준)을 통해 계층적으로 분류/예측하는 트리 구조 모델          | Decision Tree                             | 과적합되기 쉬움                        |
| Random Forest        | 여러 개의 결정 트리를 만들고 그 결과들을 종합하여 최종 예측을 수행하는 앙상블 모델        | Random Forest                             | 과적합 방지, 강력한 성능                  |
| Bootstrap Sample     | 원본 데이터셋에서 복원 추출하여 생성된 여러 개의 작은 샘플 데이터셋                 | Bootstrap Sample                          | 랜덤 포레스트의 핵심 기법                  |
| Kernel Trick         | 고차원 공간으로 데이터를 매핑하지 않고도 고차원 공간에서의 내적을 계산하는 방법           | Kernel Trick                              | SVM의 비선형 분류 능력 제공               |
| Hyperparameter       | 모델 학습 전에 사용자가 직접 설정해야 하는 매개변수                          | Hyperparameter                            | K (KNN), C (SVM), 트리 개수 (RF) 등  |
| Overfitting          | 모델이 훈련 데이터에 너무 맞춰져 새로운 데이터에 대한 예측 성능이 떨어지는 현상          | Overfitting                               | 모델 복잡도 조절로 완화                   |
| Data Leakage         | 테스트 데이터의 정보가 훈련 과정에 유출되어 모델 성능이 과대평가되는 현상              | Data Leakage                              | 교차 검증, 엄격한 데이터 분할로 방지           |
| K-fold CV            | 데이터를 K개의 폴드로 나누어 K번의 훈련/검증을 반복하는 교차 검증 기법              | K-fold Cross Validation                   | 모델 성능의 일반화된 추정                  |
| Accuracy             | 전체 예측 중 올바르게 예측한 비율                                    | Accuracy                                  | 균형 잡힌 데이터셋에 적합                  |
| Sensitivity (Recall) | 실제 양성 중 모델이 양성으로 올바르게 예측한 비율                           | Sensitivity (Recall)                      | 질병 진단 등 '놓치지 않는 것'이 중요할 때       |
| Specificity          | 실제 음성 중 모델이 음성으로 올바르게 예측한 비율                           | Specificity                               | 오탐을 줄이는 것이 중요할 때                |
| Precision            | 모델이 양성으로 예측한 것 중 실제 양성인 비율                             | Precision                                 | 스팸 메일 분류 등 '정확하게 예측하는 것'이 중요할 때 |
| Confusion Matrix     | 분류 모델의 성능을 시각화하는 표. 실제 값과 예측 값을 비교                     | Confusion Matrix                          | 모델 성능의 상세 분석                    |
| Gini Index           | 결정 트리에서 노드를 분할할 때 사용되는 불순도 측정 지표                       | Gini Index                                | 값이 낮을수록 순수도가 높음                 |
| Entropy              | 결정 트리에서 노드의 불순도를 측정하는 또 다른 지표                          | Entropy                                   | 정보 이론에서 파생                      |
| PDP                  | 특정 특징이 모델의 예측에 미치는 영향을 시각화하는 도구                        | Partial Dependence Plot                   | 블랙박스 모델 해석에 활용                  |
| SHAP                 | 특정 예측에 대한 각 특징의 기여도를 설명하는 방법                           | SHAP (SHapley Additive exPlanations)      | 블랙박스 모델 해석에 활용                  |

## 핵심 개념 및 원리

### 0.1 STM (Structural Topic Modeling) 복습

STM은 기존의 토픽 모델링 기법인 LDA(Latent Dirichlet Allocation)를 확장하여, 문서 수준의 메타데이터(공변량)를 토픽 모델링 과정에 통합하는 방법입니다. 이는 토픽의 출현 빈도(prevalence)와 토픽의 내용(content)에 메타데이터가 미치는 영향을 이해하는 데 도움을 줍니다.

#### STM의 주요 특징

- **공변량 통합:** 문서의 메타데이터(예: 작성자, 날짜, 출처)를 모델에 직접 통합합니다.
- **토픽 분포:** 토픽의 출현 빈도와 토픽 내 단어 분포에 메타데이터가 어떻게 영향을 미치는지 분석할 수 있습니다.
- **비지도 학습:** 레이블된 데이터가 필요 없는 비지도 학습 알고리즘입니다.
- **다중 토픽 소속:** 하나의 문서가 여러 토픽에 동시에 속할 수 있습니다. 이는 클러스터링과 다른 점입니다.

#### 0.1.1 STM 퀴즈 복습

강의에서는 STM에 대한 퀴즈를 통해 핵심 개념을 다시 한번 확인했습니다.

1. **STM 설명:** STM은 토픽 모델링에 공변량을 통합하여 토픽 출현 빈도에 미치는 영향을 이해하는 방법입니다. 모델은 로지스틱 정규 분포(Logistic Normal Distribution)를 사용하여 공변량을 통합하며, 이는 다변량 정규 분포에 소프트맥스를 적용한 형태입니다.
2. **텍스트 전처리 함수:** R 패키지에서 STM 분석을 위한 텍스트 전처리에는 `text_processor` 함수가 사용됩니다.
3. **메타데이터의 역할:** 문서 수준 메타데이터는 토픽 출현 빈도와 토픽 내용 추론 모두에 영향을 미칩니다. 이는 외부 요인을 설명하기 위해 모델을 조정하는 데 도움이 됩니다. 메타데이터는 토픽 시각화에도 사용될 수 있지만, 유일한 목적은 아닙니다.
4. **토픽 출현 빈도의 분포:** STM에서 토픽 출현 빈도는 로지스틱 정규 분포(Logistic Normal Distribution)를 따릅니다. 이는 다변량 정규 분포에 소프트맥스를 적용한 형태입니다. LDA(Latent Dirichlet Allocation)에서는 다항 분포(Multinomial Distribution)를 사용하지만, STM은 공변량을 통합하기 위해 로지스틱 정규 분포를 채택합니다.
5. **공변량 효과 추정 함수:** STM 패키지에서 공변량의 효과를 추정하는 데 사용되는 함수는 `estimate_effect`입니다.

## 0.2 고전적 머신러닝 기법 개요

오늘 강의에서는 데이터 과학에서 매우 중요한 고전적 머신러닝 모델들을 다룹니다. 이 모델들은 자연어 처리(NLP) 문제에도 효과적으로 적용될 수 있습니다. NLP에 적용할 때는 텍스트 데이터를 먼저 수치형 벡터 표현으로 변환하는 과정이 필수적입니다. 예를 들어, TF-IDF(Term Frequency-Inverse Document Frequency) 벡터와 같은 형태로 텍스트를 표현한 후, 이러한 벡터에 분류기(Classifier)를 적용할 수 있습니다.

### NLP에 고전적 모델 적용하기

- **텍스트 벡터화:** 문장이나 문서와 같은 텍스트 데이터를 컴퓨터가 이해할 수 있는 수치형 벡터(예: TF-IDF, Word2Vec 임베딩의 평균 등)로 변환합니다.
- **모델 적용:** 변환된 벡터를 입력으로 사용하여 분류, 회귀 등 다양한 머신러닝 작업을 수행합니다.

### 0.2.1 Naive Bayes Classifier (나이브 베이즈 분류기)

#### 한 줄 핵심 요약

나이브 베이즈 분류기는 베이즈 정리를 기반으로 하며, 모든 특징(단어)들이 주어진 클래스(스팸/정상)에 대해 서로 독립이라는 '나이브한' 가정을 사용하여 텍스트를 분류합니다.

#### 직관적 예시/비유

이메일이 스팸인지 아닌지 분류한다고 가정해 봅시다. "당첨", "무료", "클릭"이라는 단어가 포함된 이메일이 있습니다. 나이브 베이즈는 이 단어들이 서로 독립적으로 스팸일 확률에 기여한다고 가정합니다. 즉, "당첨"이 스팸일 확률, "무료"가 스팸일 확률, "클릭"이 스팸일 확률을 각각 계산한 다음, 이들을 단순히 곱하여 이메일이 스팸일 전체 확률을 추정합니다. 실제로는 단어들이 서로 연관되어 있지만, 나이브 베이즈는 이를 무시하고 단순하게 계산합니다.

#### 기술적 설명

나이브 베이즈 분류기는 베이즈 정리(Bayes' Theorem)에 기반합니다. 베이즈 정리는 조건부 확률을 뒤집어 계산하는 방법으로, 어떤 사건이 발생했을 때 다른 사건이 발생할 확률을 추론하는 데 사용됩니다.

**베이즈 정리:**  $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$

여기서:

- $P(A|B)$ :  $B$ 라는 데이터(특징 벡터)가 주어졌을 때  $A$ 라는 클래스에 속할 사후 확률(Posterior Probability)
- $P(B|A)$ :  $A$ 라는 클래스에 속할 때  $B$ 라는 데이터가 관측될 가능성도(Likelihood)
- $P(A)$ :  $A$ 라는 클래스에 속할 사전 확률(Prior Probability)
- $P(B)$ :  $B$ 라는 데이터가 관측될 확률(증거, Evidence)

**나이브 가정:** 나이브 베이즈는 특징 벡터  $X = (x_1, x_2, \dots, x_n)$ 가 주어졌을 때, 각 특징  $x_i$ 가 클래스  $C_k$ 에 대해 서로 조건부 독립이라고 가정합니다. 이 가정을 통해 가능성도  $P(X|C_k)$ 를 다음과 같이 단순화할 수 있습니다.

$$P(X|C_k) = P(x_1, x_2, \dots, x_n|C_k) = \prod_{i=1}^n P(x_i|C_k)$$

따라서, 나이브 베이즈 분류기는 다음과 같이 클래스  $C_k$ 에 속할 확률을 계산합니다.

$$P(C_k|X) = \frac{P(C_k) \cdot \prod_{i=1}^n P(x_i|C_k)}{P(X)}$$

분모  $P(X)$ 는 모든 클래스에 대해 동일하므로, 실제 분류 시에는 분자  $P(C_k) \cdot \prod_{i=1}^n P(x_i|C_k)$ 를 최대화하는 클래스를 선택합니다.

**장점:**

- **효율성:** 계산 비용이 낮아 빠릅니다.
- **해석 용이성:** 각 특징이 클래스 확률에 어떻게 기여하는지 비교적 쉽게 이해할 수 있습니다.
- **소규모 데이터셋:** 데이터가 많지 않을 때도 잘 작동할 수 있습니다.
- **스팸 탐지 및 감성 분석:** 텍스트 분류 문제에서 특히 효과적입니다.

**단점:**

- **독립성 가정:** 특징 간의 독립성 가정이 실제 데이터에서는 거의 충족되지 않아 성능에 영향을 줄 수 있습니다.
- **제로 빈도 문제:** 훈련 데이터에 없는 단어가 나타나면 해당 단어의 확률이 0이 되어 전체 확률을 0으로 만들어버리는 문제가 발생할 수 있습니다 (라플라스 스무딩 등으로 해결).

### 0.2.2 K-Nearest Neighbors (KNN, K-최근접 이웃)

#### 한 줄 핵심 요약

KNN은 새로운 데이터 포인트를 분류하거나 예측할 때, 훈련 데이터셋에서 가장 가까운 K개의 이웃들을 찾아 그들의 클래스(분류) 또는 평균 값(회귀)을 따르는 비모수적(non-parametric) 알고리즘입니다.

#### 직관적 예시/비유

새로운 고객이 어떤 상품을 구매할지 예측한다고 가정해 봅시다. KNN은 이 고객과 나이, 소득, 성별 등 여러 특성이 가장 비슷한 K명의 과거 고객들을 찾아봅니다. 만약 이 K명의 고객 중 대부분이 특정 상품을 구매했다면, 새로운 고객도 그 상품을 구매할 것이라고 예측하는 방식입니다. "친구 따라 강남 간다"는 속담처럼, 주변의 가장 가까운 이웃들의 행동을 따라가는 원리입니다.

#### 기술적 설명

KNN은 '게으른 학습(Lazy Learning)' 알고리즘으로 분류됩니다. 이는 훈련 단계에서 별도의 모델을 구축하지 않고, 모든 훈련 데이터를 저장해 두었다가 예측 시점에만 계산을 수행하기 때문입니다.

**작동 원리:**

1. **거리 계산:** 새로운 데이터 포인트와 훈련 데이터셋의 모든 포인트 간의 거리를 계산합니다.
2. **K개 이웃 선택:** 계산된 거리를 기준으로 가장 가까운 K개의 훈련 데이터 포인트를 선택합니다.
3. **예측/분류:**
  - **분류:** K개 이웃 중 가장 많은 클래스를 새로운 데이터 포인트의 클래스로 할당합니다 (다수결 투표).
  - **회귀:** K개 이웃의 목표 변수 값들의 평균을 새로운 데이터 포인트의 예측 값으로 사용합니다.

**거리 측정:** 거리 측정에는 다양한 방법이 사용될 수 있습니다.

- **유클리드 거리 (Euclidean Distance):** 가장 일반적인 거리 측정법으로, 두 점 사이의 직선 거리를 계산합니다.

- **맨해튼 거리 (Manhattan Distance):** 각 좌표축을 따라 이동하는 거리의 합을 계산합니다 (도시 블록을 이동하는 것과 유사).
- **코사인 유사도 (Cosine Similarity):** 두 벡터 간의 각도의 코사인 값을 측정하여 유사도를 판단합니다. 텍스트 데이터(TF-IDF 벡터)에서 특히 유용합니다.

**데이터 정규화의 중요성:** 유클리드 거리와 같은 거리 기반 측정법을 사용할 때는 데이터 정규화(Normalization)가 매우 중요합니다. 예를 들어, 나이(20~60세)와 소득(수천~수억 원)을 비교할 때, 소득의 스케일이 훨씬 크기 때문에 거리 계산 시 소득이 나이보다 훨씬 큰 영향을 미치게 됩니다. 모든 특징이 동등하게 기여하도록 하려면, 특징들을 동일한 스케일로 정규화해야 합니다.

**하이퍼파라미터 K:** K값은 KNN 모델의 성능에 큰 영향을 미치는 하이퍼파라미터입니다.

- **작은 K:** 노이즈에 민감하고 과적합되기 쉽습니다.
- **큰 K:** 결정 경계가 부드러워지지만, 관련 없는 이웃이 포함되어 과소적합될 수 있습니다.
- **K 선택:** 일반적으로 교차 검증(Cross-Validation)을 통해 최적의 K값을 찾습니다. 분류 문제에서는 동점(tie)을 피하기 위해 홀수 K값을 선호하는 경향이 있습니다.

**KNN의 확률론적 특성:** KNN은 겉보기에는 결정론적(deterministic) 모델처럼 보이지만, 실제로는 확률론적(stochastic)일 수 있습니다.

- **동점 발생:** 분류 문제에서 K개의 이웃 중 여러 클래스가 동률을 이룰 경우, 어떤 클래스를 선택할지 무작위로 결정해야 합니다.
- **동일 거리 이웃:** 여러 데이터 포인트가 새로운 데이터 포인트로부터 정확히 동일한 거리에 있을 경우, K개의 이웃을 선택하는 과정에서 무작위성이 개입될 수 있습니다.

이러한 이유로, 실제 구현에서는 여러 번 실행하여 평균을 내는 등의 방법을 사용하기도 합니다.

**장점:**

- **간단하고 직관적:** 구현 및 이해가 쉽습니다.
- **비선형 데이터:** 데이터가 선형적으로 분리되지 않을 때도 잘 작동할 수 있습니다.
- **훈련 단계 없음:** 모델을 미리 학습시킬 필요가 없습니다.

**단점:**

- **계산 비용:** 예측 시마다 모든 훈련 데이터와의 거리를 계산해야 하므로, 데이터셋이 클수록 계산 비용이 많이 듭니다.
- **K값 선택:** 최적의 K값을 찾는 것이 중요하며, 데이터에 따라 달라집니다.
- **차원의 저주:** 특징(차원)의 수가 많아질수록 성능이 저하될 수 있습니다.
- **데이터 불균형:** 불균형 데이터셋에서 소수 클래스에 대한 예측 성능이 낮을 수 있습니다.

### 0.2.3 Logistic Regression (로지스틱 회귀)

#### 한 줄 핵심 요약

로지스틱 회귀는 선형 회귀의 아이디어를 사용하여 입력 특징들의 선형 조합을 만든 후, 이를 시그모이드(sigmoid) 함수에 통과시켜 0과 1 사이의 확률 값을 출력함으로써 이진 분류를 수행하는 모델입니다.

**직관적 예시/비유**

어떤 학생이 시험에 합격할지 불합격할지 예측한다고 가정해 봅시다. 로지스틱 회귀는 학생의 공부 시간, 이전 성적 등 여러 요소를 종합하여 '합격 가능성 점수'를 계산합니다. 이 점수는 음수에서 양수까지 다양하게 나올 수 있습니다. 이 점수를 시그모이드 함수라는 '확률 변환기'에 넣으면, 0

**기술적 설명**

로지스틱 회귀는 이름에 '회귀'가 들어가지만 실제로는 분류(Classification) 모델입니다. 이는 선형 모델의 출력을 직접 사용하는 대신, 시그모이드 함수를 통해 확률로 변환하기 때문입니다.

**신경망과의 관계:** 로지스틱 회귀는 은닉층이 없는 가장 단순한 형태의 신경망(Shallow Network)으로 볼 수 있습니다. 입력층과 출력층만 존재하며, 출력층에서 시그모이드 활성화 함수를 사용합니다.

**모델 구조:** 입력 특징 벡터  $X = (x_1, x_2, \dots, x_k)$ 와 가중치 벡터  $W = (w_1, w_2, \dots, w_k)$ , 그리고 편향(bias)  $b$ 가 주어졌을 때, 선형 조합은 다음과 같습니다.  $z = w_1x_1 + w_2x_2 + \dots + w_kx_k + b = W^T X + b$

이  $z$  값을 시그모이드 함수  $\sigma(z)$ 에 통과시켜 0과 1 사이의 확률  $\hat{y}$ 를 얻습니다.  $\hat{y} = P(Y = 1|X) = \sigma(z) = \frac{1}{1+e^{-z}}$

**결정 경계 (Decision Boundary):** 로지스틱 회귀는 확률  $\hat{y}$ 가 특정 임계값(일반적으로 0.5)을 넘으면 클래스 1로, 그렇지 않으면 클래스 0으로 분류합니다.  $\hat{y} = 0.5$ 가 되는 지점은  $z = 0$ 이 되는 지점과 같습니다. 따라서  $W^T X + b = 0$ 이 결정 경계가 됩니다. 이는 특징 공간에서 항상 선형(직선 또는 평면) 형태를 가집니다.

**손실 함수 및 최적화:** 로지스틱 회귀는 일반적으로 최대 우도 추정(Maximum Likelihood Estimation)을 통해 모델의 가중치  $W$ 와 편향  $b$ 를 학습합니다. 이진 분류 문제에서 최대 우도 추정은 교차 엔트로피(Cross-Entropy) 손실 함수를 최소화하는 것과 수학적으로 동일합니다.

**장점:**

- **간단하고 효율적:** 구현이 쉽고 계산 비용이 낮습니다.
- **확률 제공:** 각 클래스에 속할 확률을 직접 제공하여 예측의 불확실성을 이해하는 데 도움이 됩니다.
- **해석 용이성:** 가중치  $W$ 를 통해 각 특징이 결과에 미치는 영향의 방향과 크기를 해석할 수 있습니다.

**단점:**

- **선형 결정 경계:** 데이터가 선형적으로 분리될 수 있을 때만 좋은 성능을 보입니다. 비선형 관계를 모델링하려면 특징 공학(Feature Engineering)을 통해 새로운 비선형 특징을 수동으로 추가해야 합니다.
- **과소적합:** 복잡한 데이터셋에서는 과소적합(Underfitting)될 가능성이 있습니다.

**0.2.4 Support Vector Machines (SVM, 서포트 벡터 머신)****한 줄 핵심 요약**

SVM은 두 클래스 사이의 '마진(margin)'을 최대화하는 최적의 결정 경계(초평면)를 찾아 데이터를 분류하는 강력한 지도 학습 모델입니다.

## 직관적 예시/비유

빨간색 사과와 초록색 사과가 섞여 있는 접시가 있다고 상상해 봅시다. SVM은 이 두 종류의 사과를 가장 넓은 간격으로 분리할 수 있는 선(결정 경계)을 찾습니다. 이 선은 단순히 사과들을 나누는 것을 넘어, 각 사과 그룹에서 가장 가까운 사과들(서포트 벡터)로부터 최대한 멀리 떨어져 있도록 그려집니다. 마치 두 그룹 사이에 가장 넓은 '고속도로'를 만드는 것과 같습니다.

## 기술적 설명

SVM은 분류 문제에서 특히 강력한 성능을 보이는 모델입니다. 핵심 아이디어는 두 클래스 사이의 마진을 최대화하는 결정 경계를 찾는 것입니다.

**결정 경계와 마진:** 결정 경계는 데이터를 두 클래스로 나누는 선(2차원) 또는 초평면(고차원)입니다. SVM은 이 결정 경계와 가장 가까운 훈련 데이터 포인트(서포트 벡터) 사이의 거리를 최대화하려고 합니다. 이 거리를 마진이라고 합니다. 마진이 클수록 모델의 일반화 성능이 좋다고 간주됩니다.

**서포트 벡터 (Support Vectors):** 마진을 정의하는 데 사용되는 훈련 데이터 포인트들을 서포트 벡터라고 합니다. 이들은 결정 경계에 가장 가까이 위치한 점들로, 모델의 결정 경계는 오직 이 서포트 벡터들에 의해서만 결정됩니다. 다른 훈련 데이터 포인트들은 결정 경계에 영향을 미치지 않습니다.

## 하드 마진 vs 소프트 마진:

- **하드 마진 (Hard Margin):** 모든 훈련 데이터 포인트를 완벽하게 분리하고, 마진 내에 어떤 데이터도 허용하지 않는 결정 경계를 찾습니다. 데이터가 완벽하게 선형 분리 가능할 때만 사용할 수 있으며, 이상치(outlier)에 매우 민감하여 과적합되기 쉽습니다.
- **소프트 마진 (Soft Margin):** 일부 훈련 데이터 포인트가 마진 내에 있거나 심지어 잘못 분류되는 것을 허용하면서 마진을 최대화합니다. 이는 '슬랙 변수( $\xi$ )'와 '규제 매개변수  $C$ '를 도입하여 구현됩니다.
  - **$C$  매개변수:** 오분류에 대한 패널티를 조절합니다.
    - \*  $C$ 가 작을수록: 오분류를 더 많이 허용하고 마진을 넓게 만듭니다 (과소적합 경향).
    - \*  $C$ 가 클수록: 오분류를 적게 허용하고 마진을 좁게 만듭니다 (과적합 경향).

소프트 마진은 과적합을 방지하고, 선형 분리 불가능한 데이터셋에도 적용할 수 있게 해줍니다.

**커널 트릭 (Kernel Trick):** SVM의 가장 강력한 특징 중 하나는 커널 트릭을 사용하여 비선형 결정 경계를 만들 수 있다는 것입니다.

- **문제:** 데이터가 선형적으로 분리 불가능할 때 (예: 원형으로 분포된 데이터).
- **아이디어:** 원래의 저차원 특징 공간의 데이터를 고차원 특징 공간으로 매핑하면, 저차원에서 비선형이었던 관계가 고차원에서는 선형적으로 분리될 수 있습니다.
- **커널 함수:** 실제로 데이터를 고차원으로 매핑하지 않고도, 고차원 공간에서의 내적(dot product)을 계산하는 커널 함수를 사용합니다. 이는 계산 비용을 크게 줄여줍니다.
- **주요 커널 함수:**
  - **선형 커널 (Linear Kernel):** 원래 특징 공간에서 선형 결정 경계를 만듭니다.
  - **다항식 커널 (Polynomial Kernel):** 다항식 형태의 결정 경계를 만듭니다.
  - **방사 기저 함수 (Radial Basis Function, RBF) 커널:** 가우시안 커널이라고도 하며, 원형 또는 타원형의 결정 경계를 만듭니다. 가장 널리 사용되는 커널 중 하나입니다.

## 장점:

- **강력한 성능:** 복잡한 고차원 데이터셋에서도 높은 분류 정확도를 보입니다.



- 커널 트릭: 비선형 결정 경계를 효과적으로 처리할 수 있습니다.
- 과적합 방지: 마진 최대화와 규제 매개변수  $C$ 를 통해 과적합에 강합니다.

단점:

- 계산 복잡성: 데이터셋의 크기가 매우 클 경우 훈련 시간이 오래 걸릴 수 있습니다.
- 매개변수 튜닝: 커널 함수와  $C$  매개변수 등 하이퍼파라미터 선택에 따라 성능이 크게 달라질 수 있습니다.
- 해석의 어려움: 결정 경계가 복잡해질수록 모델의 예측을 해석하기 어렵습니다.

### 0.2.5 Decision Tree (결정 트리)

#### 한 줄 핵심 요약

결정 트리는 데이터를 특정 기준에 따라 계층적으로 분할하여, 최종적으로 각 노드에 속하는 데이터의 클래스(분류) 또는 값(회귀)을 예측하는 나무 형태의 모델입니다.

#### 직관적 예시/비유

어떤 사람이 대출을 받을 수 있을지 없을지 결정하는 과정을 생각해 봅시다. 결정 트리는 "소득이 5천만원 이상인가?", "신용 등급이 700점 이상인가?"와 같은 질문들을 순차적으로 던집니다. 각 질문의 답변(예/아니오)에 따라 다음 질문으로 이동하고, 최종적으로 "대출 승인" 또는 "대출 거절"이라는 결론에 도달합니다. 마치 스무고개 게임처럼 질문을 통해 답을 찾아가는 과정입니다.

#### 기술적 설명

결정 트리는 직관적이고 이해하기 쉬운 모델로, 일련의 'if-then-else' 규칙을 통해 의사결정을 시뮬레이션합니다.

작동 원리:

1. **노드 분할**: 훈련 데이터셋을 가장 잘 분리하는 특징과 분할 기준(threshold)을 찾습니다.
2. **재귀적 분할**: 분할된 각 서브셋에 대해 1단계 과정을 재귀적으로 반복합니다.
3. **정지 조건**: 모든 데이터가 하나의 클래스에 속하거나, 더 이상 분할할 수 없거나, 미리 정의된 최대 깊이에 도달하는 등의 조건이 충족되면 분할을 멈춥니다.
4. **리프 노드**: 최종적으로 분할이 멈춘 노드를 리프 노드(leaf node)라고 하며, 이 노드에 속한 데이터의 다수결 클래스(분류) 또는 평균 값(회귀)이 예측 결과가 됩니다.

**분할 기준**: 노드를 분할할 때 어떤 특징과 기준을 선택할지는 '불순도(impurity)'를 최소화하는 방향으로 결정됩니다.

- **분류 문제**:
  - **지니 불순도 (Gini Impurity/Index)**: 노드 내에서 무작위로 선택된 두 샘플이 서로 다른 클래스에 속할 확률을 측정합니다. 값이 낮을수록 노드의 순수도가 높습니다.
  - **엔트로피 (Entropy)**: 노드 내 클래스 분포의 무질서도를 측정합니다. 정보 이득(Information Gain)을 최대화하는 방향으로 분할합니다.
- **회귀 문제**:
  - **평균 제곱 오차 (Mean Squared Error, MSE) 또는 잔차 제곱합 (Sum of Squared Errors, SSE)**: 분할 후 각 노드의 예측 값과 실제 값 사이의 오차 제곱합을 최소화하는 방향으로 분할합니다.

다.

**과적합 문제:** 결정 트리는 훈련 데이터에 완벽하게 맞춰지도록 계속 분할될 수 있습니다. 이 경우 트리가 너무 깊고 복잡해져 훈련 데이터에는 완벽하지만, 새로운 데이터에 대한 예측 성능이 떨어지는 과적합(Overfitting) 문제가 발생하기 쉽습니다.

**과적합 방지:**

- **가지치기 (Pruning):** 너무 깊게 성장한 트리의 일부 가지를 잘라내어 단순화합니다.
- **하이퍼파라미터 설정:** 트리의 최대 깊이(max\_depth), 노드 분할에 필요한 최소 샘플 수(min\_samples\_split) 등을 제한합니다.

**장점:**

- **해석 용이성:** 모델의 의사결정 과정을 시각적으로 이해하기 쉽습니다.
- **데이터 전처리 불필요:** 특징 스케일링이나 정규화가 필요하지 않습니다.
- **비선형 관계 처리:** 특징 간의 복잡한 비선형 관계를 모델링할 수 있습니다.

**단점:**

- **과적합:** 단일 결정 트리는 과적합되기 매우 쉽습니다.
- **불안정성:** 훈련 데이터의 작은 변화에도 트리의 구조가 크게 바뀔 수 있습니다.
- **최적화 문제:** 최적의 트리를 찾는 것이 NP-완전 문제이므로, 휴리스틱한 방법(탐욕 알고리즘)을 사용합니다.

## 0.2.6 Random Forest (랜덤 포레스트)

### 한 줄 핵심 요약

랜덤 포레스트는 여러 개의 독립적인 결정 트리를 생성하고, 각 트리의 예측을 종합(분류: 다수결, 회귀: 평균)하여 최종 예측을 수행함으로써 단일 결정 트리의 과적합 문제를 해결하고 성능을 향상시키는 앙상블 학습 모델입니다.

### 직관적 예시/비유

어떤 주식의 가격이 오를지 내릴지 예측한다고 가정해 봅시다. 한 명의 전문가(단일 결정 트리)에게만 의존하면 그 전문가의 편향이나 실수로 인해 잘못된 예측을 할 수 있습니다. 랜덤 포레스트는 여러 명의 전문가(수많은 결정 트리)를 고용합니다. 각 전문가는 서로 다른 정보(부트스트랩 샘플)를 보고, 제한된 관점(무작위 특징 선택)으로 예측합니다. 이 모든 전문가의 예측을 모아 다수결(또는 평균)로 최종 결정을 내리면, 한 명의 전문가보다 훨씬 정확하고 안정적인 예측을 할 수 있습니다.

### 기술적 설명

랜덤 포레스트는 '배깅(Bagging, Bootstrap Aggregating)'이라는 앙상블(Ensemble) 기법의 한 종류입니다. 여러 개의 약한 학습기(Weak Learner, 여기서는 결정 트리)를 결합하여 하나의 강력한 학습기를 만듭니다.

**작동 원리:** 랜덤 포레스트는 두 가지 주요 무작위성(Randomness)을 도입하여 여러 개의 '다양한' 결정 트리를 생성합니다.

#### 1. 부트스트랩 샘플링 (Bootstrap Sampling):

- 원본 훈련 데이터셋에서 복원 추출(replacement) 방식으로 여러 개의 서브 샘플 데이터셋을 생성합니다.
- 각 서브 샘플은 원본 데이터셋과 동일한 크기를 가지지만, 일부 데이터 포인트는 여러 번 포함될 수 있고, 일부는 전혀 포함되지 않을 수 있습니다 (평균적으로 약 37%의 데이터가 각 부트스트랩 샘플에 포함되지 않음, 이를 OOB(Out-Of-Bag) 샘플이라고 함).
- 각 결정 트리는 이 독립적인 부트스트랩 샘플을 사용하여 훈련됩니다.

## 2. 무작위 특징 선택 (Random Feature Subsetting):

- 각 노드를 분할할 때, 모든 특징(변수) 중에서 최적의 분할 특징을 찾는 대신, 무작위로 선택된 특징들의 부분집합 내에서만 최적의 분할 특징을 찾습니다.
- 일반적으로 특징 수의 제곱근( $\sqrt{p}$ ) 또는 로그( $\log_2 p$ ) 만큼의 특징을 고려합니다.
- 이로 인해 각 트리는 서로 다른 특징 조합에 집중하게 되어 다양성이 더욱 증가합니다.

### 최종 예측:

- 분류: 모든 트리의 예측 중 가장 많은 표를 얻은 클래스를 최종 예측으로 선택합니다 (다수결 투표).
- 회귀: 모든 트리의 예측 값들의 평균을 최종 예측 값으로 사용합니다.

### 과적합 방지 및 성능 향상:

- 과적합 방지: 개별 결정 트리는 과적합될 수 있지만, 여러 트리의 예측을 평균화하거나 다수결로 종합하면 개별 트리의 과적합으로 인한 노이즈와 편향이 상쇄되어 전체 모델의 과적합이 줄어듭니다.
- 안정성: 훈련 데이터의 작은 변화에도 강건하며, 안정적인 예측 성능을 제공합니다.
- 높은 정확도: 일반적으로 단일 결정 트리보다 훨씬 높은 예측 정확도를 달성합니다.

### 장점:

- 강력한 성능: 다양한 데이터셋에서 높은 예측 정확도를 보입니다.
- 과적합에 강함: 부트스트랩 샘플링과 무작위 특징 선택 덕분에 과적합 위험이 낮습니다.
- 특징 중요도: 각 특징이 모델 예측에 얼마나 중요한지 측정할 수 있습니다.
- 데이터 전처리 불필요: 결정 트리 기반이므로 특징 스케일링이 필요 없습니다.

### 단점:

- 블랙박스 모델: 여러 트리의 앙상블이므로 단일 결정 트리처럼 의사결정 과정을 시각적으로 해석하기 어렵습니다.
- 계산 비용: 많은 트리를 생성해야 하므로 훈련 시간이 오래 걸릴 수 있습니다.

**블랙박스 모델 해석 기법:** 랜덤 포레스트와 같은 블랙박스 모델의 해석력을 높이기 위한 기법들이 있습니다.

- 부분 의존성 플롯 (Partial Dependence Plot, PDP): 특정 특징의 값이 변할 때 모델의 예측이 어떻게 변하는지 평균적인 관계를 보여줍니다.
- SHAP (SHapley Additive exPlanations): 게임 이론의 Shapley 값을 기반으로, 각 특징이 특정 예측에 얼마나 기여했는지 정량적으로 설명합니다.

## 절차/방법

### 0.3 텍스트 데이터 전처리: TF-IDF Vectorizer

텍스트 데이터를 머신러닝 모델에 입력하기 위해서는 수치형 벡터로 변환하는 과정이 필수적입니다. TF-IDF(Term Frequency-Inverse Document Frequency)는 텍스트 벡터화의 가장 일반적인 방법 중 하나입니다.

#### TF-IDF Vectorizer의 역할

- **단어 빈도(TF):** 특정 단어가 문서 내에서 얼마나 자주 나타나는지.
- **역문서 빈도(IDF):** 특정 단어가 전체 문서 집합에서 얼마나 희귀한지.
- **중요도 측정:** TF와 IDF를 곱하여 단어의 중요도를 측정합니다. 자주 나타나면서도 희귀한 단어일수록 중요도가 높게 평가됩니다.
- **벡터 변환:** 각 문서를 어휘 집(vocabulary)의 모든 단어에 대한 TF-IDF 값으로 구성된 벡터로 변환합니다.

**주의사항:** TF-IDF Vectorizer는 훈련 데이터셋에만 fit되어야 합니다. 테스트 데이터셋의 정보가 훈련 과정에 유출되는 데이터 누수(Data Leakage)를 방지하기 위함입니다. fit된 Vectorizer는 훈련, 검증, 테스트 데이터셋 모두에 transform을 적용하여 일관된 벡터 표현을 얻습니다.

### 0.4 K-fold 교차 검증 (K-fold Cross Validation)

K-fold 교차 검증은 모델의 성능을 보다 신뢰성 있게 평가하고, 하이퍼파라미터를 튜닝하는 데 사용되는 강력한 기법입니다.

#### K-fold 교차 검증의 필요성

- **데이터 효율성:** 데이터셋이 크지 않을 때, 훈련/테스트 분할 방식은 특정 분할에 따라 성능이 크게 달라질 수 있습니다. K-fold는 모든 데이터를 훈련과 테스트에 한 번씩 사용함으로써 데이터 효율성을 높입니다.
- **성능 일반화:** 모델이 특정 훈련 데이터에 과적합되지 않고, 새로운 데이터에 얼마나 잘 일반화되는지 더 안정적으로 추정할 수 있습니다.
- **모델 재현성:** 고전적 머신러닝 모델(선형 회귀, SVM 등)은 훈련 데이터가 주어지면 결과가 재현 가능(reproducible)합니다. K-fold는 이러한 모델의 성능을 평가하는 데 적합합니다. (신경망과 같이 확률론적 요소가 강한 모델에는 K-fold가 덜 적합할 수 있습니다.)

**작동 원리 (K=5 예시):**

1. **데이터 분할:** 전체 데이터셋을 K개(예: 5개)의 동일한 크기의 '폴드(fold)'로 나눕니다.
2. **반복 훈련 및 평가:** K번의 반복을 수행합니다.
  - 각 반복에서, K-1개의 폴드를 훈련 데이터로 사용하고, 나머지 1개의 폴드를 테스트 데이터로 사용합니다.
  - 모델을 훈련 데이터로 학습시키고, 테스트 데이터로 성능을 평가합니다.
  - 이 과정을 K번 반복하여, 각 폴드가 정확히 한 번씩 테스트 데이터로 사용되도록 합니다.
3. **성능 집계:** K번의 평가에서 얻은 성능 지표(예: 정확도)들의 평균을 계산하여 최종 모델 성능으로

사용합니다.

### K-fold 교차 검증 (K=5) 시각화

```

+-----+
| 전체 데이터셋                                     |
+-----+
| Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
+-----+-----+-----+-----+-----+

1회차:
+-----+
| Train  | Train  | Train  | Train  | Test   | (모델 1 훈련, Fold 5 평가)
+-----+-----+-----+-----+-----+

2회차:
+-----+
| Train  | Train  | Train  | Test   | Train  | (모델 2 훈련, Fold 4 평가)
+-----+-----+-----+-----+-----+

... (반복) ...

5회차:
+-----+
| Test   | Train  | Train  | Train  | Train  | (모델 5 훈련, Fold 1 평가)
+-----+-----+-----+-----+-----+

최종 성능: (각 회차의 평가 점수 평균)

```

**최종 모델 선택:** K-fold 교차 검증은 모델의 일반화 성능을 추정하는 데 사용됩니다. 실제 서비스에 배포할 최종 모델은, K-fold를 통해 최적의 하이퍼파라미터를 찾은 후, **전체 훈련 데이터셋(또는 전체 데이터셋)**을 사용하여 다시 훈련시킨 모델을 사용합니다. 이는 모델이 가능한 한 많은 데이터를 학습하도록 하기 위함입니다.

## 0.5 하이퍼파라미터 튜닝 (Grid Search CV)

하이퍼파라미터는 모델 학습 전에 사용자가 직접 설정해야 하는 매개변수입니다. 모델의 성능에 큰 영향을 미치므로, 최적의 하이퍼파라미터를 찾는 과정이 중요합니다. Grid Search CV는 이러한 하이퍼파라미터 튜닝을 자동화하는 방법입니다.

### Grid Search CV의 작동 원리

- **매개변수 그리드:** 사용자가 튜닝하고자 하는 각 하이퍼파라미터에 대해 여러 후보 값들을 정의합니다.
- **모든 조합 탐색:** 정의된 그리드 내의 모든 하이퍼파라미터 조합에 대해 모델을 훈련하고 평가함

니다.

- **교차 검증:** 각 조합에 대한 모델 평가는 K-fold 교차 검증을 사용하여 수행됩니다.
- **최적 조합 선택:** 가장 좋은 성능을 보인 하이퍼파라미터 조합을 최종적으로 선택합니다.

**파이프라인 (Pipeline) 사용:** `sklearn.pipeline.Pipeline`은 여러 전처리 단계와 최종 모델을 하나의 객체로 묶어주는 유용한 도구입니다. K-fold 교차 검증과 함께 사용할 때 특히 강력합니다.

- **순차적 적용:** 파이프라인은 정의된 순서대로 각 단계를 적용합니다.
- **데이터 누수 방지:** K-fold 교차 검증 시, 각 폴드에 대해 전처리 단계(예: TF-IDF Vectorizer의 `fit`)가 훈련 데이터에만 적용되고 테스트 데이터에는 `transform`만 적용되도록 자동으로 관리하여 데이터 누수를 방지합니다.

## 0.6 모델 평가 지표

모델의 성능을 평가하는 데는 다양한 지표가 사용됩니다. 특히 데이터셋이 불균형할 경우, 단순히 정확도 (Accuracy) 만으로는 모델의 실제 성능을 제대로 파악하기 어렵습니다.

### 혼동 행렬 (Confusion Matrix)

혼동 행렬은 분류 모델의 성능을 시각화하는 표입니다. 실제 클래스와 모델이 예측한 클래스를 비교하여 True Positive (TP), False Positive (FP), False Negative (FN), True Negative (TN)의 네 가지 값을 보여줍니다.

| 실제 클래스            | 예측: 양성 (Positive)   | 예측: 음성 (Negative)   |
|-------------------|---------------------|---------------------|
| 실제: 양성 (Positive) | True Positive (TP)  | False Negative (FN) |
| 실제: 음성 (Negative) | False Positive (FP) | True Negative (TN)  |

- **TP (True Positive):** 실제 양성을 양성으로 올바르게 예측.
- **FP (False Positive):** 실제 음성을 양성으로 잘못 예측 (1종 오류).
- **FN (False Negative):** 실제 양성을 음성으로 잘못 예측 (2종 오류).
- **TN (True Negative):** 실제 음성을 음성으로 올바르게 예측.

주요 평가 지표:

- **정확도 (Accuracy):** 전체 예측 중 올바르게 예측한 비율.  $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$   
 – 언제 사용?: 데이터셋이 균형 잡혀 있을 때 유용합니다.  
 – 한계: 불균형 데이터셋에서는 높은 정확도가 오해를 불러일으킬 수 있습니다 (예: 99).
- **민감도 (Sensitivity) / 재현율 (Recall):** 실제 양성 중 모델이 양성으로 올바르게 예측한 비율.  $Recall = \frac{TP}{TP+FN}$   
 – 언제 사용?: '놓치지 않는 것'이 중요할 때 (예: 질병 진단에서 실제 환자를 놓치지 않는 것, 스팸 메일에서 중요한 메일을 스팸으로 분류하지 않는 것).
- **특이도 (Specificity):** 실제 음성 중 모델이 음성으로 올바르게 예측한 비율.  $Specificity = \frac{TN}{TN+FP}$   
 – 언제 사용?: '오탐을 줄이는 것'이 중요할 때 (예: 건강한 사람을 환자로 오진하지 않는 것).
- **정밀도 (Precision):** 모델이 양성으로 예측한 것 중 실제 양성인 비율.  $Precision = \frac{TP}{TP+FP}$   
 – 언제 사용?: '정확하게 예측하는 것'이 중요할 때 (예: 스팸 메일 분류에서 스팸으로 분류된 메일이 실제로 스팸일 확률).

### 불균형 데이터셋과 평가 지표

데이터셋이 불균형할 때(예: 한 클래스가 다른 클래스보다 훨씬 많을 때), 정확도만으로는 모델의 성능을 제대로 평가하기 어렵습니다. 이 경우, 민감도, 특이도, 정밀도, F1-Score(정밀도와 재현율의 조화 평균), ROC-AUC(Receiver Operating Characteristic - Area Under the Curve) 또는 PR-AUC(Precision-Recall - Area Under the Curve)와 같은 지표들을 함께 고려해야 합니다. 특히 PR-AUC는 불균형 데이터셋에서 모델 성능을 평가하는 데 매우 유용합니다.

- 강의명: CSCI89
- 주차: Lecture 9
- 교수명: [교수명]
- 목적: (학습 목적: 고전적 머신러닝 모델의 이해 및 텍스트 분류 적용)

## Contents

|       |                                          |    |
|-------|------------------------------------------|----|
| 0.1   | STM (Structural Topic Modeling) 복습       | 3  |
| 0.1.1 | STM 퀴즈 복습                                | 3  |
| 0.2   | 고전적 머신러닝 기법 개요                           | 4  |
| 0.2.1 | Naive Bayes Classifier (나이브 베이즈 분류기)     | 4  |
| 0.2.2 | K-Nearest Neighbors (KNN, K-최근접 이웃)      | 5  |
| 0.2.3 | Logistic Regression (로지스틱 회귀)            | 6  |
| 0.2.4 | Support Vector Machines (SVM, 서포트 벡터 머신) | 7  |
| 0.2.5 | Decision Tree (결정 트리)                    | 9  |
| 0.2.6 | Random Forest (랜덤 포레스트)                  | 10 |
| 0.3   | 텍스트 데이터 전처리: TF-IDF Vectorizer           | 12 |
| 0.4   | K-fold 교차 검증 (K-fold Cross Validation)   | 12 |
| 0.5   | 하이퍼파라미터 튜닝 (Grid Search CV)              | 13 |
| 0.6   | 모델 평가 지표                                 | 14 |
| 0.7   | 데이터 로드 및 탐색                              | 17 |
| 0.8   | 이진 분류 문제 설정                              | 18 |
| 0.9   | 모델 학습 및 평가 (K-fold 교차 검증 사용)             | 18 |
| 0.9.1 | Naive Bayes Classifier                   | 18 |
| 0.9.2 | K-Nearest Neighbors (KNN)                | 19 |
| 0.9.3 | Logistic Regression                      | 21 |
| 0.9.4 | Support Vector Machines (SVM)            | 22 |
| 0.9.5 | Random Forest                            | 24 |
| 0.10  | 모델 성능 비교                                 | 26 |



## 실습/코드/오류와 해결

강의에서는 BBC 뉴스 데이터셋을 사용하여 고전적 머신러닝 모델들을 텍스트 분류 문제에 적용하는 실습을 진행했습니다. 목표는 뉴스가 '기술(Tech)' 관련인지 아닌지를 분류하는 이진 분류 모델을 구축하는 것입니다.

### 0.7 데이터 로드 및 탐색

BBC 뉴스 데이터셋은 BBC\_data.csv 파일에 저장되어 있으며, 약 2,225개의 뉴스 기사와 5가지 카테고리(스포츠, 기술, 비즈니스, 엔터테인먼트, 정치)를 포함합니다.

```

1 import pandas as pd
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.model_selection import train_test_split, cross_val_score,
  GridSearchCV
4 from sklearn.naive_bayes import MultinomialNB
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.svm import SVC
8 from sklearn.ensemble import RandomForestClassifier
9 from sklearn.pipeline import Pipeline
10 from sklearn.metrics import accuracy_score, recall_score, confusion_matrix,
  make_scorer
11
12 # 데이터로드
13 df = pd.read_csv('BBC_data.csv')
14
15 # 데이터정보확인
16 print(df.info())
17 print("\카테고리별n 뉴스개수 :")
18 print(df['category'].value_counts())
19
20 # 문서길이분석예시  ()
21 df['num_words'] = df['news'].apply(lambda x: len(x.split()))
22 print("\카테고리별n 문서길이통계 :")
23 print(df.groupby('category')['num_words'].describe())

```

Listing 1: 데이터 로드 및 정보 확인

#### 데이터 탐색 결과

- 총 2,225개의 뉴스 기사.
- 5가지 카테고리(스포츠, 비즈니스, 정치, 기술, 엔터테인먼트)는 비교적 균등하게 분포되어 있습니다 (각 400 500개).
- 문서 길이(단어 수)는 평균 약 300단어이며, 최대 3,000단어에 이르는 아웃라이어도 존재합니다. 강의에서는 이러한 아웃라이어가 데이터 품질에 큰 문제를 일으키지 않는다고 판단했습니다.

## 0.8 이진 분류 문제 설정

원래 5가지 카테고리였지만, 실습에서는 '기술(Tech)' 카테고리를 1로, 나머지 카테고리를 0으로 하는 이진 분류 문제로 변환하여 진행합니다.

```
1 X = df['news'] # 뉴스기사텍스트
2 y = df['category'].apply(lambda x: 1 if x == 'tech' else 0) # 'tech'는 '1', 나머
   지는 0
3 print("\이진n 분류레이블분포 :")
4 print(y.value_counts())
```

Listing 2: 이진 분류 레이블 생성

### 레이블 분포

- 'tech' (1): 401개
- 'not tech' (0): 1,824개

이처럼 'not tech' 클래스가 'tech' 클래스보다 훨씬 많으므로, 데이터셋이 불균형하다는 점을 인지하고 정확도 외의 다른 평가 지표(민감도, 특이도, 정밀도)를 함께 고려해야 합니다.

## 0.9 모델 학습 및 평가 (K-fold 교차 검증 사용)

각 모델은 TF-IDF Vectorizer와 함께 파이프라인으로 구성되며, 5-fold 교차 검증을 사용하여 성능을 평가합니다.

### 0.9.1 Naive Bayes Classifier

```
1 # 파이프라인정의
2 pipeline_nb = Pipeline([
3     ('tfidf', TfidfVectorizer(stop_words='english')),
4     ('nb', MultinomialNB())
5 ])
6
7 # K-fold 교차검증설정
8 cv_splitter = KFold(n_splits=5, shuffle=True, random_state=42)
9
10 # 교차검증점수계산
11 nb_scores = cross_val_score(pipeline_nb, X, y, cv=cv_splitter, scoring='
    accuracy')
12 print(f"나이브 베이즈교차검증정확도 : {nb_scores.mean():.4f} 표준편
    차: {nb_scores.std():.4f}")
13
14 # 전체데이터셋으로모델훈련및예측평가 (지표계산을위해 )
15 pipeline_nb.fit(X, y)
16 y_pred_nb = pipeline_nb.predict(X)
17
18 # 평가지표계산
19 nb_accuracy = accuracy_score(y, y_pred_nb)
20 nb_recall = recall_score(y, y_pred_nb, pos_label=1) # tech (1)에 대한재현율
21 nb_specificity = recall_score(y, y_pred_nb, pos_label=0) # not tech (0)에 대한
    재현율특이도 ()
```

```

22 nb_cm = confusion_matrix(y, y_pred_nb)
23
24 print(f"나이브 베이즈정확도 : {nb_accuracy:.4f}")
25 print(f"나이브 베이즈민감도 (Recall for Tech): {nb_recall:.4f}")
26 print(f"나이브 베이즈특이도 (Specificity for Not Tech): {nb_specificity:.4f}")
27 print("나이브 베이즈혼동행렬 :\n", nb_cm)

```

Listing 3: 나이브 베이즈 분류기 학습 및 평가

### 나이브 베이즈 결과 해석

- 정확도: 약 0.8966
- 민감도 (Recall for Tech): 약 0.43
- 특이도 (Specificity for Not Tech): 약 1.00
- 혼동 행렬:

```

[[1824    0]
 [ 230   171]]

```

- 실제 'not tech' (0) 1,824개를 모두 'not tech'으로 정확히 예측했습니다 (TN=1824, FP=0).
- 실제 'tech' (1) 401개 중 171개만 'tech'으로 예측하고, 230개는 'not tech'으로 잘못 예측했습니다 (TP=171, FN=230).
- 결론: 모델이 'not tech' 클래스를 과도하게 예측하는 경향이 있습니다. 이는 'not tech' 클래스의 데이터 수가 훨씬 많기 때문에, 모델이 안전하게 'not tech'으로 예측하여 전체 정확도를 높이려는 전략을 취한 것으로 보입니다. 'tech' 뉴스를 놓치지 않는 것이 중요하다면 이 모델은 적합하지 않습니다.

### 0.9.2 K-Nearest Neighbors (KNN)

```

1 # 파이프라인정의기본 ( K=5)
2 pipeline_knn = Pipeline([
3     ('tfidf', TfidfVectorizer(stop_words='english')),
4     ('knn', KNeighborsClassifier(n_neighbors=5))
5 ])
6
7 # 교차검증점수계산
8 knn_scores = cross_val_score(pipeline_knn, X, y, cv=cv_splitter, scoring='
    accuracy')
9 print(f"KNN (K=5) 교차검증정확도 : {knn_scores.mean():.4f} 표준편
    차(: {knn_scores.std():.4f}")
10
11 # 전체데이터셋으로모델훈련및예측
12 pipeline_knn.fit(X, y)
13 y_pred_knn = pipeline_knn.predict(X)
14
15 # 평가지표계산
16 knn_accuracy = accuracy_score(y, y_pred_knn)
17 knn_recall = recall_score(y, y_pred_knn, pos_label=1)

```

```

18 knn_specificity = recall_score(y, y_pred_knn, pos_label=0)
19 knn_cm = confusion_matrix(y, y_pred_knn)
20
21 print(f"KNN (K=5) 정확도: {knn_accuracy:.4f}")
22 print(f"KNN (K=5) 민감도 (Recall for Tech): {knn_recall:.4f}")
23 print(f"KNN (K=5) 특이도 (Specificity for Not Tech): {knn_specificity:.4f}")
24 print("KNN (K=5) 혼동행렬 :\n", knn_cm)

```

Listing 4: KNN 분류기 학습 및 평가

### KNN (K=5) 결과 해석

- 정확도: 약 0.9708
- 민감도 (Recall for Tech): 약 0.9377
- 특이도 (Specificity for Not Tech): 약 0.9797
- 혼동 행렬:

```

[[1787   37]
 [  25 376]]

```

- 실제 'not tech' (0) 1,824개 중 1,787개를 정확히 예측하고, 37개를 잘못 예측했습니다 (FP=37).
- 실제 'tech' (1) 401개 중 376개를 정확히 예측하고, 25개를 잘못 예측했습니다 (FN=25).
- 결론: 나이브 베이즈에 비해 모든 지표에서 크게 향상되었습니다. 특히 'tech' 뉴스에 대한 민감도가 0.43에서 0.94로 대폭 상승하여, 'tech' 뉴스를 놓치는 경우가 현저히 줄었습니다. 특이도는 약간 감소했지만 여전히 높은 수준입니다.

**KNN 하이퍼파라미터 튜닝 (K 최적화):** Grid Search CV를 사용하여 최적의 K(n\_neighbors) 값을 찾습니다.

```

1 # 튜닝할하이퍼파라미터그리드정의
2 param_grid_knn = {
3     'knn_n_neighbors': range(1, 21) # K부터=1 까지20 시도
4 }
5
6 # GridSearchCV 설정
7 grid_search_knn = GridSearchCV(pipeline_knn, param_grid_knn, cv=cv_splitter,
8     scoring='accuracy', n_jobs=-1)
9 grid_search_knn.fit(X, y)
10
11 print(f"KNN 최적값 K: {grid_search_knn.best_params_}")
12 print(f"KNN 최적교차검증정확도 : {grid_search_knn.best_score_:.4f}")
13
14 # 최적값으로 K 모델훈련및예측
15 best_knn_pipeline = grid_search_knn.best_estimator_
16 y_pred_best_knn = best_knn_pipeline.predict(X)
17
18 # 평가지표계산
19 best_knn_accuracy = accuracy_score(y, y_pred_best_knn)

```

```

19 best_knn_recall = recall_score(y, y_pred_best_knn, pos_label=1)
20 best_knn_specificity = recall_score(y, y_pred_best_knn, pos_label=0)
21 best_knn_cm = confusion_matrix(y, y_pred_best_knn)
22
23 print(f"KNN 최적( K ) 정확도: {best_knn_accuracy:.4f}")
24 print(f"KNN 최적( K ) 민감도 (Recall for Tech): {best_knn_recall:.4f}")
25 print(f"KNN 최적( K ) 특이
    도 (Specificity for Not Tech): {best_knn_specificity:.4f}")
26 print("KNN 최적( K ) 혼동행렬 :\n", best_knn_cm)

```

Listing 5: KNN K값 최적화

### KNN (최적 K) 결과 해석

- 최적 K 값: 19
- 최적 교차 검증 정확도: 약 0.9802
- 정확도: 약 0.9802
- 민감도 (Recall for Tech): 약 0.9377 (K=5와 동일)
- 특이도 (Specificity for Not Tech): 약 0.9912 (K=5보다 향상)
- 혼동 행렬:

```

[[1808  16]
 [ 25 376]]

```

- 결론: K값을 19로 최적화한 결과, 정확도와 특이도가 소폭 향상되었습니다. 민감도는 동일하게 유지되었습니다. 이는 K값 튜닝을 통해 모델의 전반적인 성능을 개선할 수 있음을 보여줍니다.

### 0.9.3 Logistic Regression

```

1 # 파이프라인정의
2 pipeline_lr = Pipeline([
3     ('tfidf', TfidfVectorizer(stop_words='english')),
4     ('lr', LogisticRegression(max_iter=1000, solver='liblinear')) # max_iter
    증
    가
5 ])
6
7 # 교차검증점수계산
8 lr_scores = cross_val_score(pipeline_lr, X, y, cv=cv_splitter, scoring='
    accuracy')
9 print(f"로지스틱 회귀교차검증정확도 : {lr_scores.mean():.4f} 표준편
    차(: {lr_scores.std():.4f}")
10
11 # 전체데이터셋으로모델훈련및예측
12 pipeline_lr.fit(X, y)
13 y_pred_lr = pipeline_lr.predict(X)
14
15 # 평가지표계산
16 lr_accuracy = accuracy_score(y, y_pred_lr)

```

```

17 lr_recall = recall_score(y, y_pred_lr, pos_label=1)
18 lr_specificity = recall_score(y, y_pred_lr, pos_label=0)
19 lr_cm = confusion_matrix(y, y_pred_lr)
20
21 print(f"로지스틱 회귀정확도 : {lr_accuracy:.4f}")
22 print(f"로지스틱 회귀민감도 (Recall for Tech): {lr_recall:.4f}")
23 print(f"로지스틱 회귀특이도 (Specificity for Not Tech): {lr_specificity:.4f}")
24 print("로지스틱 회귀혼동행렬 :\n", lr_cm)

```

Listing 6: 로지스틱 회귀 분류기 학습 및 평가

### 로지스틱 회귀 결과 해석

- 정확도: 약 0.9083
- 민감도 (Recall for Tech): 약 0.7107
- 특이도 (Specificity for Not Tech): 약 0.9507
- 혼동 행렬:

```

[[1734  90]
 [ 116 285]]

```

- 결론: 나이브 베이즈보다는 민감도가 향상되었지만, KNN보다는 낮은 성능을 보입니다. 특이도는 여전히 높은 편이지만, 'not tech' 클래스에 대한 편향이 남아있습니다. 로지스틱 회귀는 선형 결정 경계를 사용하므로, 데이터의 복잡한 비선형 관계를 포착하는 데 한계가 있을 수 있습니다.

## 0.9.4 Support Vector Machines (SVM)

```

1 # 파이프라인정의기본 ( 커널='rbf', C=1)
2 pipeline_svm = Pipeline([
3     ('tfidf', TfidfVectorizer(stop_words='english')),
4     ('svc', SVC(kernel='rbf', C=1, random_state=42))
5 ])
6
7 # 교차검증점수계산
8 svm_scores = cross_val_score(pipeline_svm, X, y, cv=cv_splitter, scoring='
    accuracy')
9 print(f"SVM 기본() 교차검증정확도 : {svm_scores.mean():.4f} 표준편
    차(: {svm_scores.std():.4f}")
10
11 # 전체데이터셋으로모델훈련및예측
12 pipeline_svm.fit(X, y)
13 y_pred_svm = pipeline_svm.predict(X)
14
15 # 평가지표계산
16 svm_accuracy = accuracy_score(y, y_pred_svm)
17 svm_recall = recall_score(y, y_pred_svm, pos_label=1)
18 svm_specificity = recall_score(y, y_pred_svm, pos_label=0)
19 svm_cm = confusion_matrix(y, y_pred_svm)
20

```

```

21 print(f"SVM 기본() 정확도: {svm_accuracy:.4f}")
22 print(f"SVM 기본() 민감도 (Recall for Tech): {svm_recall:.4f}")
23 print(f"SVM 기본() 특이도 (Specificity for Not Tech): {svm_specificity:.4f}")
24 print("SVM 기본() 혼동행렬 :\n", svm_cm)

```

Listing 7: SVM 분류기 학습 및 평가(기본 설정)

### SVM (기본 설정) 결과 해석

- 정확도: 약 0.9802
- 민감도 (Recall for Tech): 약 0.8903
- 특이도 (Specificity for Not Tech): 약 0.9978
- 혼동 행렬:

```

[[1820    4]
 [  44 357]]

```

- 결론: 기본 SVM(RBF 커널)은 KNN과 유사하게 높은 정확도를 보입니다. 특이도는 거의 100

**SVM 하이퍼파라미터 튜닝 (커널 및 C 최적화):** Grid Search CV를 사용하여 최적의 커널(kernel)과 C 값을 찾습니다.

```

1 # 튜닝할하이퍼파라미터그리드정의
2 param_grid_svm = {
3     'svc__C': [0.1, 1, 10],
4     'svc__kernel': ['linear', 'rbf', 'poly']
5 }
6
7 # GridSearchCV 설정
8 grid_search_svm = GridSearchCV(pipeline_svm, param_grid_svm, cv=cv_splitter,
9     scoring='accuracy', n_jobs=-1)
10 grid_search_svm.fit(X, y)
11
12 print(f"SVM 최적하이퍼파라미터 : {grid_search_svm.best_params_}")
13 print(f"SVM 최적교차검증정확도 : {grid_search_svm.best_score_:.4f}")
14
15 # 최적하이퍼파라미터로모델훈련및예측
16 best_svm_pipeline = grid_search_svm.best_estimator_
17 y_pred_best_svm = best_svm_pipeline.predict(X)
18
19 # 평가지표계산
20 best_svm_accuracy = accuracy_score(y, y_pred_best_svm)
21 best_svm_recall = recall_score(y, y_pred_best_svm, pos_label=1)
22 best_svm_specificity = recall_score(y, y_pred_best_svm, pos_label=0)
23 best_svm_cm = confusion_matrix(y, y_pred_best_svm)
24
25 print(f"SVM 최적() 정확도: {best_svm_accuracy:.4f}")
26 print(f"SVM 최적() 민감도 (Recall for Tech): {best_svm_recall:.4f}")
27 print(f"SVM 최적() 특이

```

```

도 (Specificity for Not Tech): {best_svm_specificity:.4f}")
27 print("SVM 최적() 혼동행렬 :\n", best_svm_cm)

```

Listing 8: SVM 커널 및 C값 최적화

### SVM (최적) 결과 해석

- 최적 하이퍼파라미터: C=10, kernel='linear'
- 최적 교차 검증 정확도: 약 0.9901
- 정확도: 약 0.9901
- 민감도 (Recall for Tech): 약 0.9676
- 특이도 (Specificity for Not Tech): 약 0.9978
- 혼동 행렬:

```

[[1820    4]
 [  13 388]]

```

- 결론: C=10, linear 커널을 사용했을 때 가장 좋은 성능을 보였습니다. 정확도는 0.9901로 매우 높고, 민감도 (0.9676)와 특이도 (0.9978) 모두 우수합니다. 이는 SVM이 이 데이터셋에서 매우 강력한 분류기임을 보여줍니다.

### 0.9.5 Random Forest

```

1 # 파이프라인정의기본 ( 설정 )
2 pipeline_rf = Pipeline([
3     ('tfidf', TfidfVectorizer(stop_words='english')),
4     ('rf', RandomForestClassifier(n_estimators=100, random_state=42, criterion
5     = 'gini', max_features='sqrt'))
6 ])
7
8 # 교차검증점수계산
9 rf_scores = cross_val_score(pipeline_rf, X, y, cv=cv_splitter, scoring='
10 accuracy')
11 print(f"랜덤 포레스트기본 () 교차검증정확도 : {rf_scores.mean():.4f} 표준편
12 차: {rf_scores.std():.4f}")
13
14 # 전체데이터셋으로모델훈련및예측
15 pipeline_rf.fit(X, y)
16 y_pred_rf = pipeline_rf.predict(X)
17
18 # 평가지표계산
19 rf_accuracy = accuracy_score(y, y_pred_rf)
20 rf_recall = recall_score(y, y_pred_rf, pos_label=1)
21 rf_specificity = recall_score(y, y_pred_rf, pos_label=0)
22 rf_cm = confusion_matrix(y, y_pred_rf)
23
24 print(f"랜덤 포레스트기본 () 정확도: {rf_accuracy:.4f}")
25 print(f"랜덤 포레스트기본 () 민감도 (Recall for Tech): {rf_recall:.4f}")

```



```

23 print(f"랜덤 포레스트기본 () 특이
    도 (Specificity for Not Tech): {rf_specificity:.4f}")
24 print("랜덤 포레스트기본 () 혼동행렬 :\n", rf_cm)

```

Listing 9: 랜덤 포레스트 분류기 학습 및 평가(기본 설정)

### 랜덤 포레스트 (기본 설정) 결과 해석

- 정확도: 약 0.9802
- 민감도 (Recall for Tech): 약 0.8903
- 특이도 (Specificity for Not Tech): 약 0.9978
- 혼동 행렬:

```

[[1820    4]
 [  44 357]]

```

- 결론: 기본 랜덤 포레스트는 SVM 기본 설정과 동일한 정확도, 민감도, 특이도를 보였습니다. 이는 이 데이터셋에서 여러 모델이 유사하게 높은 성능을 달성할 수 있음을 시사합니다.

**Random Forest 하이퍼파라미터 튜닝:** Grid Search CV를 사용하여 최적의 `n_estimators`, `criterion`, `max_features`를 찾습니다.

```

1 # 튜닝할하이퍼파라미터그리드정의
2 param_grid_rf = {
3     'rf__n_estimators': [50, 100, 200],
4     'rf__criterion': ['gini', 'entropy'],
5     'rf__max_features': ['sqrt', 'log2', None] # 은None 모든특징사용
6 }
7
8 # GridSearchCV 설정
9 grid_search_rf = GridSearchCV(pipeline_rf, param_grid_rf, cv=cv_splitter,
10     scoring='accuracy', n_jobs=-1)
11
12 grid_search_rf.fit(X, y)
13
14 print(f"랜덤 포레스트최적하이퍼파라미터 : {grid_search_rf.best_params_}")
15 print(f"랜덤 포레스트최적교차검증정확도 : {grid_search_rf.best_score_:.4f}")
16
17 # 최적하이퍼파라미터로모델훈련및예측
18 best_rf_pipeline = grid_search_rf.best_estimator_
19 y_pred_best_rf = best_rf_pipeline.predict(X)
20
21 # 평가지표계산
22 best_rf_accuracy = accuracy_score(y, y_pred_best_rf)
23 best_rf_recall = recall_score(y, y_pred_best_rf, pos_label=1)
24 best_rf_specificity = recall_score(y, y_pred_best_rf, pos_label=0)
25 best_rf_cm = confusion_matrix(y, y_pred_best_rf)
26
27 print(f"랜덤 포레스트최적 () 정확도: {best_rf_accuracy:.4f}")
28 print(f"랜덤 포레스트최적 () 민감도 (Recall for Tech): {best_rf_recall:.4f}")

```

```

27 print(f"랜덤 포레스트최적 () 특이
    도 (Specificity for Not Tech): {best_rf_specificity:.4f}")
28 print("랜덤 포레스트최적 () 혼동행렬 :\n", best_rf_cm)

```

Listing 10: 랜덤 포레스트 하이퍼파라미터 최적화

### 랜덤 포레스트 (최적) 결과 해석

- 최적 하이퍼파라미터: criterion='gini', max\_features='sqrt', n\_estimators=200
- 최적 교차 검증 정확도: 약 0.9802 (기본 설정과 동일)
- 정확도: 약 0.9802
- 민감도 (Recall for Tech): 약 0.8903
- 특이도 (Specificity for Not Tech): 약 0.9978
- 혼동 행렬:

```

[[1820    4]
 [  44 357]]

```

- 결론: 하이퍼파라미터 튜닝 후에도 기본 설정과 동일한 성능을 보였습니다. 이는 기본 설정이 이미 이 데이터셋에 대해 상당히 좋은 성능을 내고 있었거나, 탐색한 하이퍼파라미터 범위 내에서 더 큰 개선이 어려웠음을 의미합니다.

## 0.10 모델 성능 비교

다음 표는 각 모델의 최종 성능 지표를 요약한 것입니다.

Table 1: 다양한 고전적 분류 모델 성능 비교

| 모델                     | 정확도 (Accuracy) | 민감도 (Recall for Tech) | 특이도 (Specificity for Not Tech) | 비고               |
|------------------------|----------------|-----------------------|--------------------------------|------------------|
| Naive Bayes            | 0.8966         | 0.4264                | 1.0000                         | 'not tech' 편향 심함 |
| KNN (K=5)              | 0.9708         | 0.9377                | 0.9797                         | 균형 잡힌 성능         |
| KNN (K=19, 최적)         | 0.9802         | 0.9377                | 0.9912                         | 특이도 향상           |
| Logistic Regression    | 0.9083         | 0.7107                | 0.9507                         | 나이브 베이즈보다 개선     |
| SVM (기본, RBF)          | 0.9802         | 0.8903                | 0.9978                         | 높은 특이도           |
| SVM (최적, Linear, C=10) | <b>0.9901</b>  | <b>0.9676</b>         | <b>0.9978</b>                  | 전반적으로 가장 우수      |
| Random Forest (기본)     | 0.9802         | 0.8903                | 0.9978                         | SVM 기본과 유사       |
| Random Forest (최적)     | 0.9802         | 0.8903                | 0.9978                         | 기본과 동일           |

### 모델 비교 결론

- 최고 성능: SVM (최적화된 Linear 커널, C=10) 모델이 정확도, 민감도, 특이도 모든 면에서 가장 우수한 성능을 보였습니다. 특히 민감도가 0.9676으로 'tech' 뉴스를 거의 놓치지 않으면서도, 특이도 또한 매우 높게 유지했습니다.
- 나이브 베이즈의 한계: 나이브 베이즈는 'not tech' 클래스에 대한 과도한 편향으로 인해 'tech' 뉴스를 많이 놓치는 한계를 보였습니다.
- KNN의 강점: KNN은 하이퍼파라미터 튜닝을 통해 매우 좋은 성능을 달성했으며, 특히 'tech' 뉴스에 대한 높은 민감도를 보여주었습니다.
- 랜덤 포레스트: 랜덤 포레스트도 높은 성능을 보였지만, 이 데이터셋에서는 SVM 최적화 모델만

크의 개선은 이루지 못했습니다.  
이 결과는 SVM이 2012년 신경망의 부상 이전까지 가장 강력한 모델 중 하나였음을 다시 한번 입증합니다.